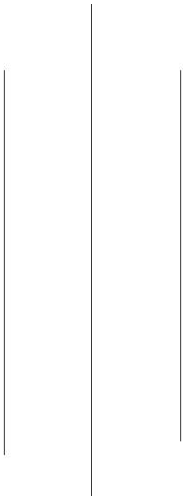
NATIONAL COLLEGE OF COMPUTER STUDIES (NCCS)

Paknajol, Kathmandu



LAB ASSIGNMENT

ON

Data Warehousing and Data Mining

Submitted By:

Krishna Gharti

BIM 7th Semester

Section: A

Roll No: 10

Submitted To:

Mr. Nabaraj Bhadur Negi

Lab 1: Install Python 3 (Anaconda) for lab preparation.

Aim: To install Python 3 and the Anaconda distribution for performing data analysis and machine learning experiments.

Software/Tools Required

- Windows Operating System
- Anaconda Distribution
- Jupyter Notebook

Theory

Anaconda is an open-source distribution of Python and R used for data science, machine learning, and scientific computing. It comes with many pre-installed libraries such as:

- numpy
- pandas
- matplotlib
- scikit-learn
- jupyter notebook

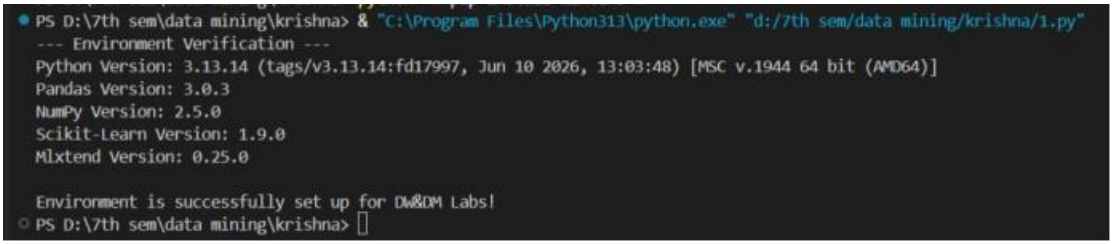
Anaconda also provides **Anaconda Navigator**, which helps users launch applications such as Jupyter Notebook and Spyder easily.

Source Code

```
import sys

import pandas as pd

import numpy as np

import sklearn

import mlxtend



print("--- Environment Verification ---")

print(f"Python Version: {sys.version}")

print(f"Pandas Version: {pd.__version__}")

print(f"NumPy Version: {np.__version__}")
```

```
print(f"Scikit-Learn Version: {sklearn.__version__}")

print(f"Mlxtend Version: {mlxtend.__version__}")

print("\nEnvironment is successfully set up for DW&DM Labs!")
```

Output



Conclusion

Python 3 and Anaconda were successfully installed. The environment is now ready for performing data mining and machine learning experiments using Jupyter Notebook and Python libraries.

Lab 2: Demonstrate the concept of data preprocessing using Python libraries.

Aim: To perform data preprocessing operations such as handling missing values, encoding categorical data, and feature scaling using Python libraries.

Theory

Data preprocessing is the process of converting raw data into a clean and understandable format before applying data mining or machine learning algorithms.

The main preprocessing techniques are:

1. Handling missing values
2. Encoding categorical data
3. Feature scaling (Normalization/Standardization)

Source Code

```
import pandas as pd

import numpy as np

from sklearn.preprocessing import LabelEncoder, MinMaxScaler



# Create a sample dataset

data = {

    'Name': ['Ram', 'Sita', 'Hari', 'Gita', 'Shyam'],

    'Age': [22, np.nan, 25, 23, np.nan],

    'City': ['Kathmandu', 'Pokhara', 'Kathmandu', 'Butwal', 'Pokhara'],

    'Salary': [30000, 40000, 35000, 45000, 50000]

}



df = pd.DataFrame(data)



print("Original Dataset:")
```

```
print(df)

# Handling missing values
df['Age'] = df['Age'].fillna(df['Age'].mean())

print("\nDataset after handling missing values:")
print(df)

# Encoding categorical data
encoder = LabelEncoder()
df['City'] = encoder.fit_transform(df['City'])

print("\nDataset after encoding:")
print(df)

# Feature Scaling
scaler = MinMaxScaler()
df[['Age', 'Salary']] = scaler.fit_transform(df[['Age', 'Salary']])

print("\nDataset after normalization:")
print(df)
```

Output

```
PS D:\7th sem\data mining\krishna> & "C:\Program Files\Python313\python.exe" "d:/7th sem/data mining/krishna/2.py"
Original Dataset:
    Name   Age       City  Salary
0    Ram  22.0  Kathmandu   30000
1   Sita   NaN    Pokhara   40000
2   Hari  25.0  Kathmandu   35000
3   Gita  23.0     Butwal   45000
4  Shyam   NaN    Pokhara   50000

Dataset after handling missing values:
    Name        Age       City  Salary
0    Ram  22.000000  Kathmandu   30000
1   Sita  23.333333    Pokhara   40000
2   Hari  25.000000  Kathmandu   35000
3   Gita  23.000000     Butwal   45000
4  Shyam  23.333333    Pokhara   50000

Dataset after encoding:
    Name        Age  City  Salary
0    Ram  22.000000     1   30000
1   Sita  23.333333     2   40000
2   Hari  25.000000     1   35000
3   Gita  23.000000     0   45000
4  Shyam  23.333333     2   50000

Dataset after normalization:
    Name       Age  City  Salary
0    Ram  0.000000     1    0.00
1   Sita  0.444444     2    0.50
2   Hari  1.000000     1    0.25
3   Gita  0.333333     0    0.75
4  Shyam  0.444444     2    1.00
PS D:\7th sem\data mining\krishna>
```

Conclusion

Data preprocessing is an essential step in data mining and machine learning. It improves the quality of data by handling missing values, converting categorical data into numerical form, and scaling features for better model performance.

Lab 3: Write a program to implement Apriori algorithm and generate association rules.

Aim: To implement the Apriori algorithm in Python and generate association rules using:

- Minimum Support = **50%**
- Minimum Confidence = **60%**

Theory

- **Support:**

  It measures how frequently an itemset appears in the dataset.

$$Support(A) = \frac{\text{Transactions containing A}}{\text{Total Transactions}}$$

- **Confidence:**

  It measures the likelihood that item B is purchased when item A is purchased.

$$Confidence(A \rightarrow B) = \frac{Support(A \cup B)}{Support(A)}$$

- **Lift:**

  It measures the strength of a rule over random occurrence.

$$Lift = \frac{Confidence(A \rightarrow B)}{Support(B)}$$

Source Code

```
import pandas as pd

from mlxtend.preprocessing import TransactionEncoder

from mlxtend.frequent_patterns import apriori, association_rules
```

```
# Transaction dataset
dataset = [
    ['Apple', 'Banana', 'Milk'],
    ['Apple', 'Diaper', 'Beer', 'Eggs'],
    ['Milk', 'Diaper', 'Beer', 'Cola'],
    ['Apple', 'Banana', 'Milk', 'Diaper'],
    ['Banana', 'Milk', 'Diaper', 'Beer']
]

# Convert transactions into one-hot encoding
te = TransactionEncoder()
te_array = te.fit(dataset).transform(dataset)
df = pd.DataFrame(te_array, columns=te.columns_)

print("Transaction Data:")
print(df)

# Generate frequent itemsets
frequent_itemsets = apriori(
    df,
    min_support=0.5,
    use_colnames=True
)
```

```
print("\nFrequent Itemsets:")
print(frequent_itemsets)

# Generate association rules
rules = association_rules(
    frequent_itemsets,
    metric="confidence",
    min_threshold=0.6
)

print("\nAssociation Rules:")
print(rules[['antecedents',
             'consequents',
             'support',
             'confidence',
             'lift']])
```

Output

```
PS D:\7th sem\data mining\krishna> & "C:\Program Files\Python313\python.exe" "d:/7th sem/data mining/krishna/3.py"
Transaction Data:
   Apple  Banana   Beer   Cola  Diaper   Eggs   Milk
0   True    True  False  False   False  False   True
1   True   False   True  False    True   True  False
2  False   False   True   True    True  False   True
3   True    True  False  False    True  False   True
4  False    True   True  False    True  False   True

Frequent Itemsets:
   support                itemsets
0      0.6        frozenset({Apple})
1      0.6       frozenset({Banana})
2      0.6         frozenset({Beer})
3      0.8       frozenset({Diaper})
4      0.8         frozenset({Milk})
5      0.6  frozenset({Banana, Milk})
6      0.6  frozenset({Diaper, Beer})
7      0.6  frozenset({Diaper, Milk})

Association Rules:
          antecedents          consequents  support  confidence    lift
0  frozenset({Banana})    frozenset({Milk})      0.6        1.00  1.2500
1    frozenset({Milk})  frozenset({Banana})      0.6        0.75  1.2500
2  frozenset({Diaper})    frozenset({Beer})      0.6        0.75  1.2500
3    frozenset({Beer})  frozenset({Diaper})      0.6        1.00  1.2500
4  frozenset({Diaper})    frozenset({Milk})      0.6        0.75  0.9375
5    frozenset({Milk})  frozenset({Diaper})      0.6        0.75  0.9375
PS D:\7th sem\data mining\krishna>
```

Conclusion

The Apriori algorithm successfully generated frequent itemsets and association rules from the transaction dataset. It is widely used in **market basket analysis**, recommendation systems, and decision-making applications to identify relationships among items.

Lab 4: Write a program to implement FP-Growth algorithm and generate association rules.

Aim: To implement the FP-Growth (Frequent Pattern Growth) algorithm in Python and generate association rules using:

- Minimum Support = **50%**
- Minimum Confidence = **60%**

Theory

FP-Growth Algorithm

FP-Growth (Frequent Pattern Growth) is a data mining algorithm used to find frequent itemsets without generating candidate itemsets like the Apriori algorithm.

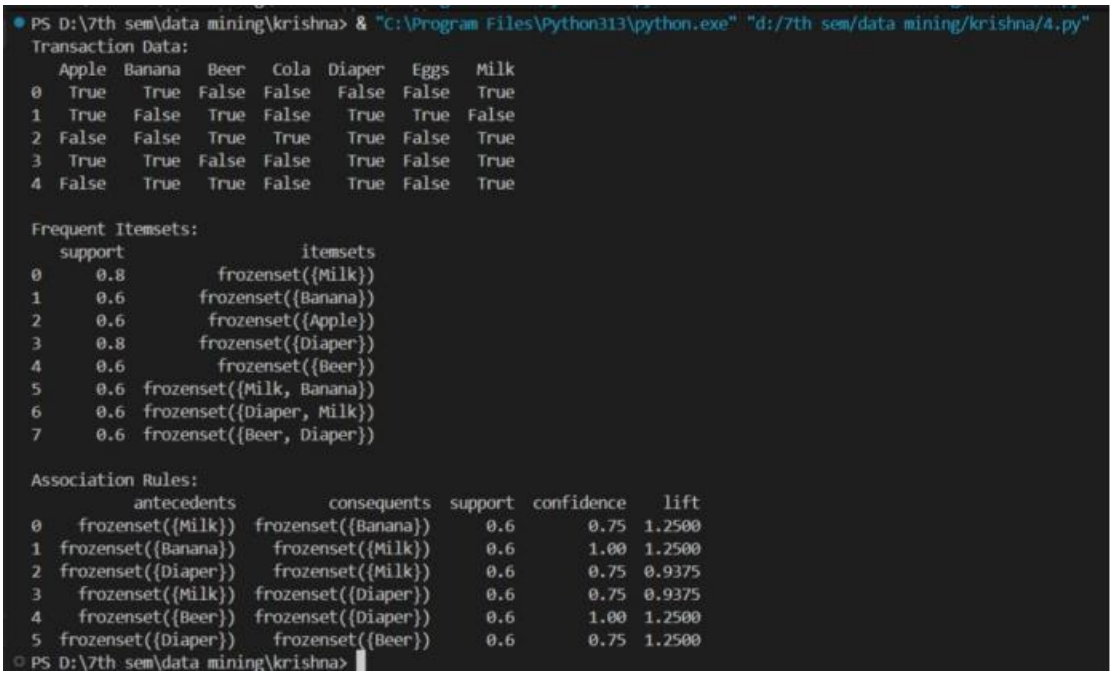
Source Code

```
import pandas as pd

from mlxtend.preprocessing import TransactionEncoder

from mlxtend.frequent_patterns import fpgrowth, association_rules



# Transaction dataset

dataset = [

    ['Apple', 'Banana', 'Milk'],

    ['Apple', 'Diaper', 'Beer', 'Eggs'],

    ['Milk', 'Diaper', 'Beer', 'Cola'],

    ['Apple', 'Banana', 'Milk', 'Diaper'],

    ['Banana', 'Milk', 'Diaper', 'Beer']

]



# Convert transactions into one-hot encoding

te = TransactionEncoder()
```

```
te_array = te.fit(dataset).transform(dataset)
df = pd.DataFrame(te_array, columns=te.columns_)

print("Transaction Data:")
print(df)

# Generate frequent itemsets using FP-Growth
frequent_itemsets = fpgrowth(
    df,
    min_support=0.5,
    use_colnames=True
)

print("\nFrequent Itemsets:")
print(frequent_itemsets)

# Generate association rules
rules = association_rules(
    frequent_itemsets,
    metric="confidence",
    min_threshold=0.6
)

print("\nAssociation Rules:")
```

```
print(rules[['antecedents',
        'consequents',
        'support',
        'confidence',
        'lift']])
```

Output



```
PS D:\7th sem\data mining\krishna> & "C:\Program Files\Python313\python.exe" "d:/7th sem/data mining/krishna/4.py"
Transaction Data:
   Apple  Banana   Beer   Cola  Diaper   Eggs   Milk
0   True    True  False  False   False  False   True
1   True   False   True  False    True   True  False
2  False   False   True   True    True  False   True
3   True    True  False  False    True  False   True
4  False    True   True  False    True  False   True

Frequent Itemsets:
   support                itemsets
0      0.8         frozenset({Milk})
1      0.6       frozenset({Banana})
2      0.6        frozenset({Apple})
3      0.8       frozenset({Diaper})
4      0.6         frozenset({Beer})
5      0.6  frozenset({Milk, Banana})
6      0.6  frozenset({Diaper, Milk})
7      0.6  frozenset({Beer, Diaper})

Association Rules:
          antecedents         consequents  support  confidence    lift
0    frozenset({Milk})  frozenset({Banana})      0.6        0.75  1.2500
1  frozenset({Banana})    frozenset({Milk})      0.6        1.00  1.2500
2  frozenset({Diaper})    frozenset({Milk})      0.6        0.75  0.9375
3    frozenset({Milk})  frozenset({Diaper})      0.6        0.75  0.9375
4    frozenset({Beer})  frozenset({Diaper})      0.6        1.00  1.2500
5  frozenset({Diaper})    frozenset({Beer})      0.6        0.75  1.2500
PS D:\7th sem\data mining\krishna>
```

Conclusion

The FP-Growth algorithm successfully generated frequent itemsets and association rules from the transaction dataset. It is more efficient than the Apriori algorithm because it uses an FP-Tree structure and avoids candidate itemset generation, making it suitable for large transactional databases.

Lab 5: Write a program to demonstrate classification using the ID3 algorithm.

Aim: To implement the ID3 (Iterative Dichotomiser 3) algorithm in Python for classification.

Theory

The ID3 algorithm is a decision tree classification algorithm developed by Ross Quinlan.

It builds a decision tree by selecting the attribute with the highest Information Gain at each node.

Algorithm

1. Calculate the entropy of the dataset.
2. Compute the information gain of each attribute.
3. Select the attribute with the highest information gain.
4. Create a decision node.
5. Repeat until all records belong to the same class.

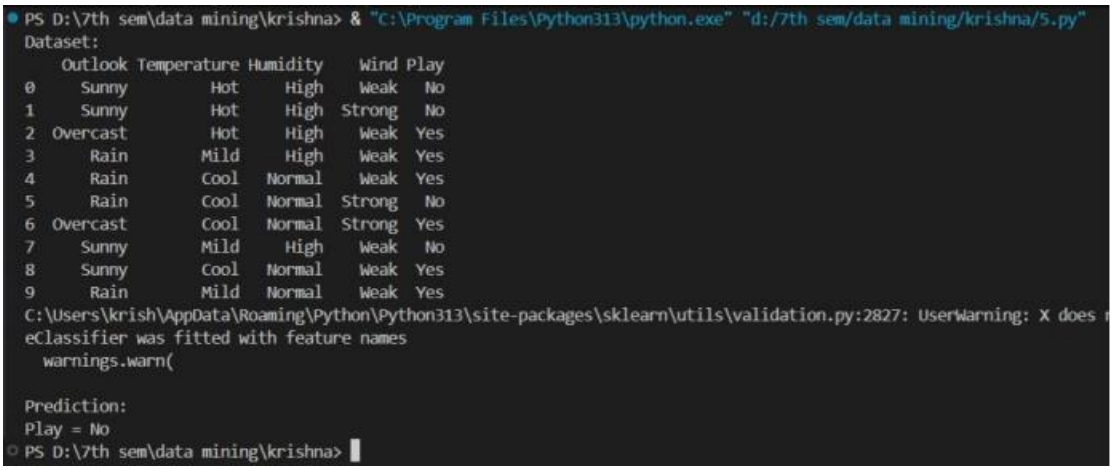
Source Code

```
import pandas as pd

from sklearn.preprocessing import LabelEncoder

from sklearn.tree import DecisionTreeClassifier



# Sample dataset

data = {

    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain',

                'Rain', 'Overcast', 'Sunny', 'Sunny', 'Rain'],

    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool',

                    'Cool', 'Cool', 'Mild', 'Cool', 'Mild'],

    'Humidity': ['High', 'High', 'High', 'High', 'Normal',

                 'Normal', 'Normal', 'High', 'Normal', 'Normal'],

    'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak',
```

```
            'Strong', 'Strong', 'Weak', 'Weak', 'Weak'],
    'Play': ['No', 'No', 'Yes', 'Yes', 'Yes',
            'No', 'Yes', 'No', 'Yes', 'Yes']
}

df = pd.DataFrame(data)

print("Dataset:")
print(df)

# Convert categorical data into numerical values
le = LabelEncoder()

for column in df.columns:
    df[column] = le.fit_transform(df[column])

X = df.drop('Play', axis=1)
y = df['Play']

# ID3 uses entropy
model = DecisionTreeClassifier(criterion='entropy')
model.fit(X, y)

# Prediction
```

```
prediction = model.predict([[2, 2, 1, 1]])


print("\nPrediction:")

if prediction[0] == 1:

    print("Play = Yes")

else:

    print("Play = No")
```

Output



Conclusion

The ID3 algorithm successfully classified the data by constructing a decision tree using entropy and information gain. It is widely used in data mining and machine learning for solving classification problems.

Lab 6: Write a program to demonstrate classification using the Bayesian algorithm.

Aim: To implement the Naive Bayes Classification Algorithm using Python and classify data.

Theory

The Naive Bayes Algorithm is a probabilistic classification algorithm based on Bayes' Theorem.

Algorithm

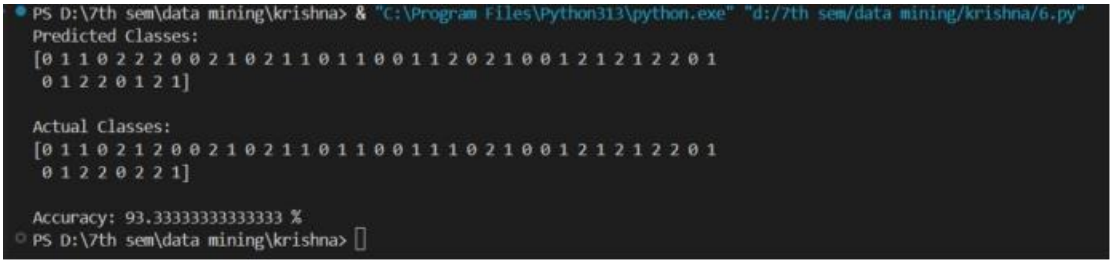
1. Load the dataset.
2. Split the data into training and testing sets.
3. Create the Naive Bayes classifier.
4. Train the model.
5. Predict the class labels.
6. Calculate the accuracy.

Source Code

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score



# Load dataset

iris = load_iris()



X = iris.data

y = iris.target



# Split dataset

X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.3, random_state=1
)

# Create model
model = GaussianNB()

# Train model
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Predicted Classes:")
print(y_pred)

print("\nActual Classes:")
print(y_test)

print("\nAccuracy:", accuracy * 100, "%")
```

Output



Conclusion

The Naive Bayes algorithm successfully classified the dataset with high accuracy. It is simple, fast, and widely used for:

- Email spam filtering
- Document classification
- Sentiment analysis
- Medical diagnosis

Lab 7: Write a program to demonstrate classification using the K-NN algorithm.

Aim: To implement the K-Nearest Neighbors (K-NN) algorithm in Python and classify data.

Theory

The K-Nearest Neighbors (K-NN) algorithm is a supervised machine learning algorithm used for classification and regression.

It classifies a new data point based on the majority class of its K nearest neighbors**.**

Steps of K-NN Algorithm

1. Choose the value of **K**.
2. Calculate the distance between the new data point and all training data points.
3. Select the K nearest neighbors.
4. Find the majority class among the neighbors.
5. Assign the majority class to the new data point.

Algorithm

1. Load the dataset.
2. Split the dataset into training and testing sets.
3. Create the K-NN classifier.
4. Train the model.
5. Predict the class labels.
6. Calculate the accuracy.

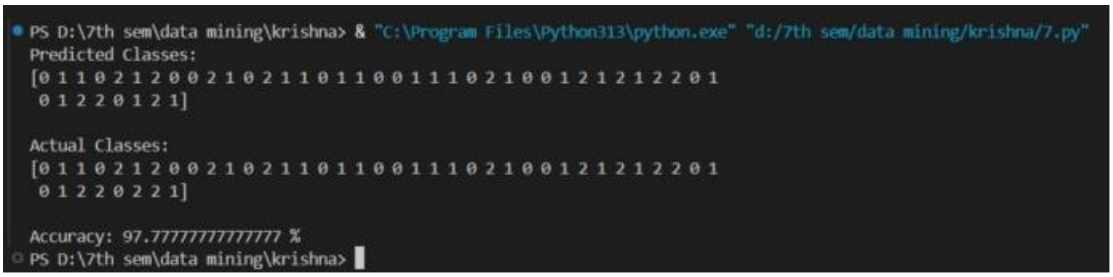
Source Code

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score



# Load Iris dataset

iris = load_iris()



X = iris.data
```

```
y = iris.target

# Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=1
)

# Create K-NN model
model = KNeighborsClassifier(n_neighbors=3)

# Train the model
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Predicted Classes:")
print(y_pred)
```

```
print("\nActual Classes:")

print(y_test)


print("\nAccuracy:", accuracy * 100, "%")
```

Output



Conclusion

The K-Nearest Neighbors (K-NN) algorithm successfully classified the dataset with high accuracy. It is simple, easy to implement, and widely used in:

- Pattern recognition
- Recommendation systems
- Medical diagnosis
- Image classification

Lab 8: Write a program to demonstrate classification using the SVM algorithm.

Aim: To implement the Support Vector Machine (SVM) algorithm in Python and classify data.

Theory

The Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression problems.

SVM works by finding the optimal hyperplane that separates the data points of different classes with the maximum margin.

Algorithm

1. Load the dataset.
2. Split the dataset into training and testing sets.
3. Create the SVM classifier.
4. Train the model.
5. Predict the class labels.
6. Calculate the accuracy.

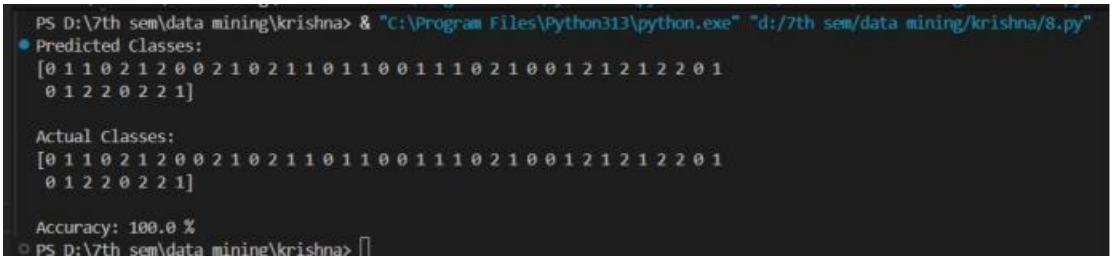
Source Code

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score



# Load Iris dataset

iris = load_iris()



X = iris.data

y = iris.target



# Split dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)

# Create SVM model
model = SVC(kernel='linear')

# Train the model
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Predicted Classes:")
print(y_pred)

print("\nActual Classes:")
print(y_test)
```

print("\nAccuracy:", accuracy * 100, "%")

Output



Conclusion

The Support Vector Machine (SVM) algorithm successfully classified the dataset with high accuracy. SVM is one of the most powerful classification algorithms and is widely used in machine learning applications such as image recognition, text classification, and medical diagnosis.

Lab 9: Write a program to implement Linear Regression algorithm.

Aim: To implement the Linear Regression algorithm in Python and predict the dependent variable from the independent variable.

Theory

Linear Regression is a supervised machine learning algorithm used to predict a continuous value based on the relationship between variables.

Algorithm

1. Import the required libraries.
2. Create the dataset.
3. Split the dataset into training and testing data.
4. Train the Linear Regression model.
5. Predict the output.
6. Display the regression equation and prediction.

Source Code

```
import numpy as np

from sklearn.linear_model import LinearRegression



# Independent variable

X = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)



# Dependent variable

y = np.array([2, 4, 6, 8, 10])



# Create model

model = LinearRegression()



# Train model
```

```
model.fit(X, y)


# Predict value for X = 6

prediction = model.predict([[6]])


print("Coefficient (Slope):", model.coef_[0])

print("Intercept:", model.intercept_)

print("Predicted value for X = 6:", prediction[0])
```

Output

Conclusion

The Linear Regression algorithm successfully learned the relationship between the independent and dependent variables and predicted the output value accurately. It is widely used for prediction and forecasting problems.

Lab 10: Write a program to implement Multiple Linear Regression algorithm.

Aim: To implement the Multiple Linear Regression algorithm in Python and predict the dependent variable using multiple independent variables.

Theory

Multiple Linear Regression (MLR) is an extension of Simple Linear Regression that uses two or more independent variables to predict a dependent variable.

Algorithm

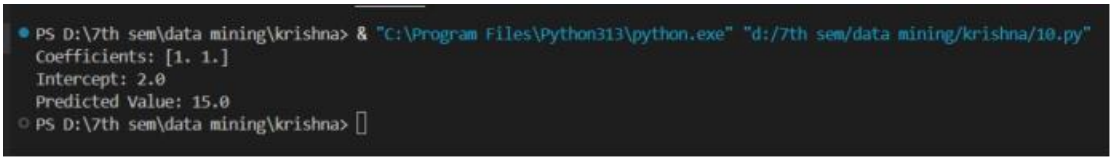
1. Import the required libraries.
2. Create the dataset.
3. Define independent and dependent variables.
4. Create the Multiple Linear Regression model.
5. Train the model using the dataset.
6. Predict the output for new data.

Source Code

```
import numpy as np

from sklearn.linear_model import LinearRegression



# Independent variables (X1 and X2)

X = np.array([

    [1, 2],

    [2, 3],

    [3, 4],

    [4, 5],

    [5, 6]

])



# Dependent variable

y = np.array([5, 7, 9, 11, 13])
```

```
# Create model

model = LinearRegression()


# Train the model

model.fit(X, y)


# Predict for X1=6 and X2=7

prediction = model.predict([[6, 7]])


print("Coefficients:", model.coef_)

print("Intercept:", model.intercept_)

print("Predicted Value:", prediction[0])
```

Output



Conclusion

The Multiple Linear Regression algorithm successfully predicted the dependent variable using two independent variables. It is widely used in prediction and forecasting problems where multiple factors affect the outcome.

Lab 11: Write a program to demonstrate clustering using the K-Means algorithm.

Aim: To implement the K-Means Clustering Algorithm in Python and group the data into clusters.

Theory

K-Means is an unsupervised machine learning algorithm used to divide data into K clusters based on similarity.

Steps of K-Means Algorithm

1. Choose the number of clusters (**K**).
2. Select K initial centroids.
3. Assign each data point to the nearest centroid.
4. Calculate new centroids.
5. Repeat steps 3 and 4 until the centroids do not change.

Algorithm

1. Import the required libraries.
2. Create the dataset.
3. Choose the number of clusters.
4. Apply the K-Means algorithm.
5. Display the cluster labels and centroids.

Source Code

```
import numpy as np

from sklearn.cluster import KMeans



# Dataset

X = np.array([

    [1, 2],

    [1, 4],

    [2, 3],

    [5, 8],

    [6, 8],

    [6, 9]
```

```
])


# Create K-Means model

kmeans = KMeans(n_clusters=2, random_state=0)


# Train the model

kmeans.fit(X)


# Cluster labels

print("Cluster Labels:")

print(kmeans.labels_)


# Centroids

print("\nCluster Centroids:")

print(kmeans.cluster_centers_)
```

Output

Conclusion

The K-Means algorithm successfully grouped the dataset into two clusters and calculated their centroids. It is widely used in:

- Customer segmentation
- Image compression
- Market basket analysis
- Pattern recognition

Lab 12: Write a program to demonstrate clustering using the K-Modes algorithm.

Aim: To implement the K-Modes clustering algorithm in Python and cluster categorical data.

Theory

K-Modes is an extension of the K-Means algorithm designed for categorical data**.**

Unlike K-Means:

- K-Means uses mean and Euclidean distance.
- K-Modes uses mode and matching dissimilarity.

Steps of K-Modes Algorithm

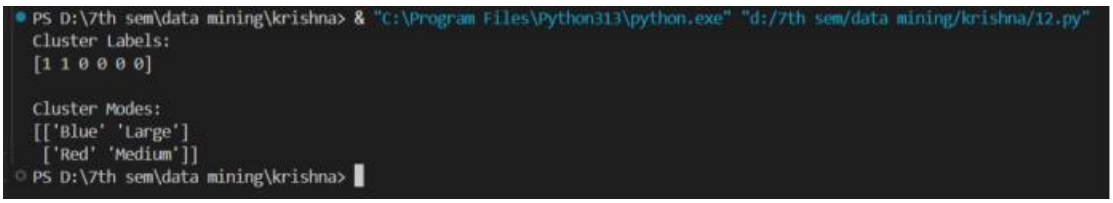
1. Select the number of clusters (**K**).
2. Choose initial modes.
3. Assign each data point to the nearest mode.
4. Update the modes.
5. Repeat until the modes do not change.

Source Code

```
import numpy as np

from kmodes.kmodes import KModes



# Categorical dataset

X = np.array([

    ['Red', 'Small'],

    ['Red', 'Medium'],

    ['Blue', 'Small'],

    ['Blue', 'Large'],

    ['Red', 'Large'],

    ['Blue', 'Medium']

])
```

```
# Create K-Modes model
km = KModes(n_clusters=2,
            init='Huang',
            n_init=5,
            random_state=42)

# Fit model and predict clusters
clusters = km.fit_predict(X)

print("Cluster Labels:")
print(clusters)

print("\nCluster Modes:")
print(km.cluster_centroids_)
```

Output



Conclusion

The K-Modes algorithm successfully clustered the categorical dataset into two groups. It is particularly useful when the dataset contains non-numeric (categorical) attributes**,** where K-Means cannot be directly applied.

Lab 13: Write a program to demonstrate clustering using the K-Medoids algorithm.

Aim: To implement the K-Medoids clustering algorithm in Python and group data into clusters.

Theory

K-Medoids is a clustering algorithm similar to K-Means. The difference is:

- K-Means uses the mean (centroid) of a cluster.
- K-Medoids uses an actual data point (medoid) as the center of a cluster.

Because it uses actual data points, K-Medoids is more robust to noise and outliers**.**

Steps of K-Medoids Algorithm

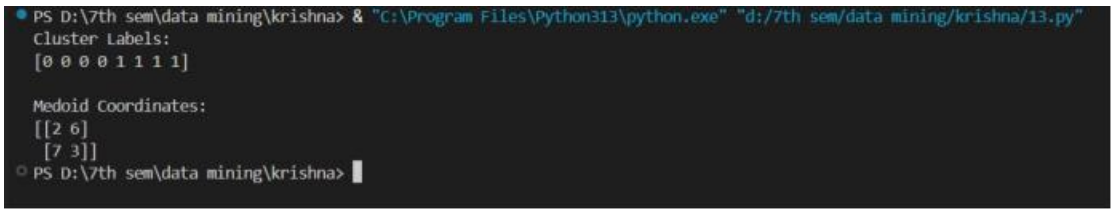
1. Choose the number of clusters (**K**).
2. Select K data points as initial medoids.
3. Assign each point to the nearest medoid.
4. Update medoids to minimize total distance.
5. Repeat until the medoids do not change.

Source Code

```
import numpy as np

from sklearn_extra.cluster import KMedoids



# Dataset

X = np.array([

    [2, 6],

    [3, 4],

    [3, 8],

    [4, 7],

    [6, 2],

    [7, 3],

    [7, 4],

    [8, 5]
```

```
])


# Create K-Medoids model
kmedoids = KMedoids(
    n_clusters=2,
    random_state=42
)


# Train the model
kmedoids.fit(X)


# Display results
print("Cluster Labels:")
print(kmedoids.labels_)
 print("\nMedoid Coordinates:")
print(kmedoids.cluster_centers_)
```

Output



Conclusion

The K-Medoids algorithm successfully clustered the dataset into two groups using actual data points as cluster centers. It is more resistant to noise and outliers than K-Means and is widely used in data mining and pattern recognition.

Lab 14: Write a program to demonstrate clustering using the Agglomerative algorithm.

Aim: To implement the Agglomerative Hierarchical Clustering Algorithm in Python and group data into clusters.

Theory

Agglomerative Clustering is a **bottom-up hierarchical clustering algorithm**.

- Initially, each data point is considered as a separate cluster.
- The two nearest clusters are merged repeatedly.
- The process continues until the desired number of clusters is obtained.

Algorithm

1. Start with each data point as a separate cluster.
2. Calculate the distance between all clusters.
3. Merge the two closest clusters.
4. Repeat until the required number of clusters is reached.

Source Code

```
import numpy as np

from sklearn.cluster import AgglomerativeClustering



# Dataset

X = np.array([

    [1, 2],

    [1, 4],

    [2, 3],

    [5, 8],

    [6, 8],

    [6, 9]

])
```

```
# Create Agglomerative model

model = AgglomerativeClustering(

    n_clusters=2,

    linkage='ward'

)



# Fit and predict clusters

labels = model.fit_predict(X)



print("Cluster Labels:")

print(labels)
```

Output

Conclusion

The Agglomerative algorithm successfully grouped similar data points into hierarchical clusters. It is useful in:

- Customer segmentation
- Document clustering
- Image segmentation
- Biological data analysis
- Pattern recognition

Lab 15: Write a program to demonstrate clustering using the Divisive algorithm.

Aim: To implement the Divisive Hierarchical Clustering Algorithm in Python and group data into clusters.

Theory

Divisive Clustering is a top-down hierarchical clustering algorithm.

- Initially, all data points belong to a single cluster.
- The algorithm repeatedly divides the clusters into smaller clusters until the desired number of clusters is obtained.

Since Scikit-Learn does not provide a direct Divisive Clustering algorithm, we can demonstrate it using Bisecting K-Means, which follows the divisive approach.

Algorithm

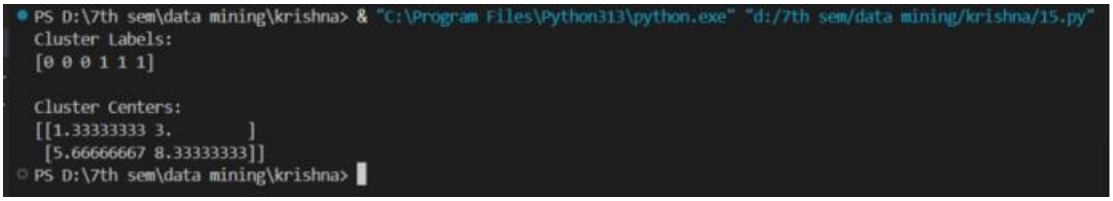
1. Start with all data points in one cluster.
2. Divide the cluster into two sub-clusters using K-Means.
3. Select one cluster and divide it again.
4. Continue until the required number of clusters is obtained.

Source Code

```
import numpy as np

from sklearn.cluster import KMeans



# Dataset

X = np.array([

    [1, 2],

    [1, 4],

    [2, 3],

    [5, 8],

    [6, 8],

    [6, 9]

])
```

```
# First split using K-Means (Divisive approach)
kmeans = KMeans(
    n_clusters=2,
    random_state=42,
    n_init=10
)

labels = kmeans.fit_predict(X)

print("Cluster Labels:")
print(labels)

print("\nCluster Centers:")
print(kmeans.cluster_centers_)
```

Output



Conclusion

The Divisive Clustering algorithm starts with a single cluster and recursively splits it into smaller clusters. In Python, it can be demonstrated using the Bisecting K-Means approach, which follows the top-down hierarchical clustering concept.

Lab 16: Write a program to demonstrate clustering using the DBSCAN algorithm.

Aim: To implement the DBSCAN (Density-Based Spatial Clustering of Applications with Noise) algorithm in Python and identify clusters and noise points.

Theory

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a density-based clustering algorithm that groups together closely packed points and identifies isolated points as noise (outliers).

Algorithm

1. Choose eps and min_samples.
2. Select an unvisited point.
3. Find all neighboring points within eps.
4. Create a new cluster if sufficient neighbors exist.
5. Continue until all points are visited.

Source Code

```
import numpy as np

from sklearn.cluster import DBSCAN



# Dataset

X = np.array([

    [1, 2],

    [2, 2],

    [2, 3],

    [8, 7],

    [8, 8],

    [25, 80]

])



# Create DBSCAN model
```

```
dbscan = DBSCAN(
    eps=3,
    min_samples=2
)

# Fit the model
labels = dbscan.fit_predict(X)

print("Cluster Labels:")
print(labels)
```

Output

Conclusion

The DBSCAN algorithm successfully grouped dense regions into clusters and identified outliers as noise. It is widely used in:

- Anomaly detection
- Spatial data analysis
- Image processing
- Customer segmentation
- Geographic data clustering